

Prüfungsleistung: Data Analytics Engine und Testautomatisierung für den Mayfly Algorithm

Studiengang: Angewandte Informatik · **Modul:** Advanced Software Engineering

Prüfungsart: Programmmentwurf

Maximalpunktzahl: 100 P

Inhalt

1. [Lernziele](#)
 2. [Rahmenbedingungen](#)
 3. [Ausgangslage und gestellte Materialien](#)
 4. [KI-Nutzungsrichtlinie \(50/50-Regel\)](#)
 5. [Aufgabenstellung](#)
 - [Phase 1 — ohne KI-Unterstützung \(50 P\)](#)
 - [Aufgabe 1 — Refactoring & Data Analytics Engine \(35 P\)](#)
 - [Aufgabe 2 — JUnit-5-Test-Suite \(15 P\)](#)
 - [Phase 2 — mit KI-Unterstützung \(50 P\)](#)
 - [Aufgabe 3 — JGiven-Akzeptanztests \(25 P\)](#)
 - [Aufgabe 4 — Reporting, Export & statistische Auswertung \(15 P\)](#)
 - [Aufgabe 5 — Dokumentation & KI-Reflexion \(10 P\)](#)
 6. [Punkteübersicht](#)
 7. [Bewertungs- und Qualitätskriterien](#)
 8. [Abgabeformat](#)
 9. [Wichtige Hinweise](#)
 10. [Anhang A — Glossar](#)
-

1. Lernziele

Nach erfolgreicher Bearbeitung dieser Prüfung können die Studierenden:

- einen bestehenden, statisch strukturierten Algorithmus durch geeignetes **Refactoring** observierbar machen, ohne dessen wissenschaftliches Verhalten zu verändern;
 - eine **Data Analytics Engine** mit klaren Interfaces und Trennung der Belange (SoC) für die Dimensionen *Agenten-Interaktion*, *globales Gedächtnis*, *lokales Gedächtnis* und *Konvergenz* entwerfen und implementieren;
 - eine **mehrstufige Teststrategie** (Unit/Integration/Acceptance) mit JUnit 5, AssertJ und JGiven aufbauen;
 - moderne, **stabile Java-25-LTS-Sprachfeatures** professionell einsetzen;
 - den **Einsatz generativer KI** im Software-Engineering kritisch reflektieren und sauber dokumentieren.
-

2. Rahmenbedingungen

Aspekt	Vorgabe
Programmiersprache	Java 25 LTS , keine Preview-Features (<code>--enable-preview</code> ist verboten)
Erlaubte Bibliotheken	Lombok (keine experimentellen Annotationen), JUnit 5 (Jupiter) , AssertJ , JGiven
Verbotene Bibliotheken	Alle weiteren Drittbibliotheken (insb. Mockito, Spring, Apache Commons, Guava, etc.)
Build-Tool	Maven oder Gradle (Begründung im README, eines der beiden)
Concurrency	Nicht erlaubt (single-threaded; weder <code>Thread</code> / <code>ExecutorService</code> noch parallele Streams)
Versionskontrolle	Git mit nachvollziehbarer Commit-Historie (mind. 1 Commit pro Teilaufgabe)
JDK	OpenJDK / Temurin 25 (Build-Datei muss <code>release</code> bzw. <code>sourceCompatibility = 25</code> setzen)
Mindest-Testabdeckung	≥ 80 % Branch-Coverage über Produktivklassen (JaCoCo, im Build verankert)
Sprache der Artefakte	Code & Doku deutsch oder englisch — innerhalb eines Artefakts konsistent

3. Ausgangslage und gestellte Materialien

Gegeben ist eine lauffähige Basisimplementierung des **Mayfly Algorithm (MA)**

zur Minimierung der **Ackley-Funktion** in 10 Dimensionen im Paket

`edu.swarmintelligence.mayfly`. Die Implementierung besteht aus vier Klassen:

Klasse	Rolle
<code>MayflyOptimizerConfig</code>	Konfigurationsrecord mit allen Hyperparametern und Factory-Methode <code>ackley10D()</code>
<code>Mayfly</code>	Datenklasse eines einzelnen Agenten (<code>pos[]</code> , <code>vel[]</code> , <code>pbestPos[]</code> , <code>fitness</code> , <code>pbestFitness</code>) inkl. <code>updatePersonalBest()</code>
<code>MayflyOptimizer</code>	Kernklasse; führt den vollständigen Algorithmus in <code>optimize()</code> aus
<code>Main</code>	Einstiegspunkt; erzeugt Default-Config und startet den Optimizer

Die Basisimplementierung realisiert:

- vollständige Initialisierung männlicher und weiblicher Mayfly-Populationen (je `populationSize` Agenten);
- synchrone Geschwindigkeits- und Positionsupdates der Männchen inkl. *nuptial dance* für das beste Männchen;

- rangbasierte Paarung weiblicher und männlicher Agenten mit Anziehungs- bzw. Random-Walk-Update;
- Crossover (Blend, Parameter $L \sim U(0, 1)$) und Gauss-Mutation aller Offspring-Dimensionen;
- globale $(\mu + \lambda)$ -Selektion über den gemeinsamen Pool aus Males, Females und Offspring;
- lineares Inertia-Weight-Schedule ($w_{\max} \rightarrow w_{\min}$);
- Konsolaausgabe des Fortschritts per `System.out.printf` direkt im Algorithmus-Code.

Bekannte Schwachstellen der Basisimplementierung (Ziel des Refactorings in Aufgabe 1.1):

- `MayflyOptimizer`: Seed `42L` ist im Konstruktor hart verdrahtet — der Algorithmus ist von außen nicht deterministisch steuerbar.
- `MayflyOptimizer`: `gbestPosition`, `gbestFitness` und `rng` sind Instanzfelder; nach `optimize()` bleibt Zustand erhalten — eine erneute Ausführung auf derselben Instanz würde korrumpierte Ergebnisse liefern.
- `MayflyOptimizer`: Zwei `System.out.printf`-Aufrufe (Initialmeldung vor der Schleife und Fortschrittsmeldung alle 100 Iterationen) sind mitten in der Algorithmuslogik eingebettet — Output und Berechnungslogik sind nicht getrennt; eine Datenerfassung ohne Konsolenausgabe ist nicht möglich.
- `MayflyOptimizerConfig`: Der `record`-Kompaktkonstruktor fehlt — keine Validierung der Hyperparameter (z. B. `dimensions > 0`, `lowerBound < upperBound`, `populationSize > 1`).
- `Main.main()`: Nicht-idiomatische Signatur — `static void main()` ist in Java 25 technisch ein valider Einstiegspunkt (JEP 495, seit Java 24 finalisiert; gilt auch für benannte Klassen mit Paketdeklaration). Gefordert ist dennoch die kanonische Form `public static void main(String[] args)` für Klarheit, Werkzeugkompatibilität und Konventionalität.

Wichtig: Die wissenschaftliche Korrektheit des Algorithmus selbst ist *nicht* Gegenstand der Bewertung. Sämtliche Modifikationen müssen verhaltensgleich oder dokumentiert äquivalent sein (siehe [Aufgabe 1.1](#)).

4. KI-Nutzungsrichtlinie (50/50-Regel)

Die Prüfung gliedert sich in zwei klar getrennte Phasen. Die **Phase ist pro Aufgabe fix zugewiesen** und darf nicht getauscht werden.

Phase 1 — ohne KI (50 P)

Erlaubt	Verboten
IDE inkl. nicht-KI-basierter Code-Completion	Code-Completion durch LLMs (Copilot, Cursor AI, JetBrains AI, Tabnine, ...)
Offizielle Java-/Bibliotheks-Dokumentation	ChatGPT, Claude, Gemini, Le Chat, Perplexity, ...
Eigene Notizen, Vorlesungsunterlagen	Übersetzung von Aufgabentexten via LLM
Stack-Overflow zum Lesen	Stack-Overflow-Antworten kopieren

Selbstverpflichtung: Der Abgabe ist eine ausgefüllte und handschriftlich unterschriebene `HONOR_DECLARATION.md` (als eingescanntes PDF oder Foto, eingebettet in die Datei oder als Beilage `HONOR_DECLARATION_signed.pdf`) beizufügen, die bestätigt, dass Phase 1 ohne KI-Unterstützung bearbeitet wurde. Die Verletzung dieser Regel führt zur Bewertung der gesamten Prüfungsleistung mit 0 Punkten.

Phase 2 — mit KI (50 P)

- Genau **ein** KI-Tool nach Wahl ist zu deklarieren (Modellname, Version, Hersteller, Datum).
- **Vollständiges Prompt-Log** (Eingaben *und* relevante Ausgaben) ist in `docs/ai-usage-log.md` abzulegen — chronologisch, pro Aufgabe getrennt, mit Zeitstempeln.
- Übernommener KI-Code ist im Source per `// AI-generated:` zu markieren und im Log zu referenzieren.
- Die Bewertung in Phase 2 setzt eigenständige **kritische Prüfung** des KI-Outputs voraus — blind übernommener, fehlerhafter Code wird gewertet wie selbst geschriebener fehlerhafter Code.

5. Aufgabenstellung

Phase 1 — ohne KI-Unterstützung (50 P)

Aufgabe 1 — Refactoring & Data Analytics Engine (35 P)

1.1 Refactoring der MA-Implementierung (5 P)

Die Basisimplementierung ist so umzubauen, dass sie von außen analysierbar und reproduzierbar wird, **ohne** das wissenschaftliche Ergebnis zu verändern.

Pflichtanforderungen

- Umbenennung: `MayflyOptimizer` → `MayflyAlgorithm`; `MayflyOptimizerConfig` → `MayflyConfig`.
- `MayflyAlgorithm` wird zur zustandslosen Klasse: `gbestPosition`, `gbestFitness` und `rng` werden aus den Instanzfeldern herausgelöst und sind nur noch lokal innerhalb von `run()` sichtbar.
- Seed von außen steuerbar: `MayflyResult run(MayflyConfig cfg, long seed)` erzeugt den `RandomGenerator (Xoshiro256PlusPlus)` ausschließlich aus dem übergebenen `seed`.
- **Rückgabotyp** `MayflyResult` ist als unveränderlicher Record zu definieren:

```
record MayflyResult(double[] gbestPosition, double gbestFitness)
```

`gbestPosition` enthält eine defensive Kopie des finalen globalen Bestpositions-Vektors; `gbestFitness` den zugehörigen Fitnesswert. Der Record wird im selben Paket wie `MayflyAlgorithm` abgelegt.

- `MayflyConfig` (bisher `MayflyOptimizerConfig`) erhält einen kompakten Konstruktor mit Validierung aller Parameter (z. B. `dimensions > 0`, `lowerBound < upperBound`, `populationSize > 1`, `wMin < wMax`, Koeffizienten `> 0`).
- `Main` wird zu einem validen Java-Einstiegspunkt korrigiert:
`public static void main(String[] args).`

- Beide `System.out.printf`-Aufrufe werden aus `MayflyAlgorithm` entfernt und durch die Listener-/Observer-API ersetzt (s. u.).
- Listener-/Observer-API: `MayflyEventListener` als funktionales Interface mit der Methode `void onEvent(MayflyEvent e)`; `MayflyAlgorithm` bietet `addListener(MayflyEventListener l)` an. Event-Typen mindestens für:
`IterationStarted`, `MaleUpdated`, `FemaleUpdated`, `OffspringCreated`,
`PbestUpdated`, `GbestUpdated`, `IterationCompleted`, `RunCompleted`.
- Events sind als **sealed interface** `MayflyEvent` mit `record`-Implementierungen zu modellieren.
- Jeder Event-Record muss mindestens die folgenden Felder tragen, damit die Analyzer (1.3–1.6) ihre Metriken berechnen können:

Event-Typ	Pflicht-Felder (Minimum)
<code>IterationStarted</code>	<code>int iteration</code> , <code>double inertiaWeight</code>
<code>MaleUpdated</code>	<code>Mayfly agent</code> , <code>boolean isNuptialDance</code> , <code>double previousFitness</code>
<code>FemaleUpdated</code>	<code>Mayfly agent</code> , <code>boolean isAttracted</code> , <code>double previousFitness</code>
<code>OffspringCreated</code>	<code>Mayfly offspring</code>
<code>PbestUpdated</code>	<code>Mayfly agent</code> , <code>double previousPbestFitness</code> , <code>double newPbestFitness</code>
<code>GbestUpdated</code>	<code>UpdateSource source</code> , <code>double previousGbestFitness</code> , <code>double newGbestFitness</code>
<code>IterationCompleted</code>	<code>int iteration</code> , <code>double gbestFitness</code> , <code>List<Mayfly> survivors</code>
<code>RunCompleted</code>	<code>MayflyResult result</code>

`UpdateSource` ist ein Enum mit den Werten `MALE`, `FEMALE`, `OFFSPRING`.

`IterationCompleted.survivors` enthält die $2 \times \text{populationSize}$ überlebenden Agenten nach globaler Selektion (für Diversitätsmetriken in 1.5 und 1.6).

Hinweis — Event-Scope: Events werden ausschließlich innerhalb der Iterationsschleife gefeuert. Die initiale gbest-Ermittlung in `optimize()` (die Schleife über die Anfangspopulationen vor Iteration 1) löst **kein** `GbestUpdated`-Event aus; sie ist eine Vorbedingung des Laufs. Der initiale gbest-Wert fließt ab Iteration 1 in `gbestTrajectory` (1.4) und `convergenceCurve` (1.6) ein. Ein `INITIALIZATION`-Wert im `UpdateSource`-Enum ist damit nicht erforderlich und darf **nicht** ergänzt werden.

Verhaltensgleichheit

Bei identischem Seed muss der Refactoring-Output bitidentisch zur Basisimplementierung sein (Toleranz: 0,0 — gleicher `RandomGenerator`-Typ `xoshiro256PlusPlus`, gleiche Aufruf-Reihenfolge). Abweichungen sind im Architektur-Dokument **wissenschaftlich** zu rechtfertigen und gelten als zusätzliche Anforderung an die Tests.

1.2 Analytics-Infrastruktur (5 P)

- `AnalyzerResult` als Marker-Interface (keine Methoden); alle konkreten Analyzer-Ergebnistypen implementieren es.
- Top-Level-Interface `MayflyAnalyzer` extends `MayflyEventListener`, das die geerbte Methode `onEvent(MayflyEvent e)` um `AnalyzerResult result()` und `String name()` erweitert.
- Aggregator `AnalyticsEngine`, der mehrere `MayflyAnalyzer`-Instanzen registriert und sequenziell versorgt (ohne Threads). `AnalyticsEngine` implementiert selbst `MayflyEventListener` und wird via `MayflyAlgorithm.addListener(...)` registriert; eingehende Events leitet sie der Reihe nach an alle registrierten Analyzer weiter.
- Ergebnis-Aggregat als `record AnalyticsReport(Map<String, AnalyzerResult> byAnalyzer, Instant generatedAt, MayflyConfig config, long seed)`.
Der Schlüssel der Map entspricht dem Rückgabewert von `MayflyAnalyzer.name()`.
- Korrekte Trennung von Daten-Erfassung (Listener / `onEvent`) und Daten-Auswertung (`result()`).

1.3 Agenten-Interaktions-Analyzer (7 P)

Zu erfassen und auszuwerten sind mindestens:

Metrik	Definition
<code>nuptialDanceCount</code>	Anzahl Nuptial-Dances über alle Iterationen
<code>attractionCount</code>	Anzahl Anziehungs-Updates der Männchen über alle Iterationen, ohne Nuptial-Dance-Updates (d. h. Updates des besten Männchens per Random Walk sind ausgenommen)
<code>femaleAttractionRate</code>	Anteil weiblicher Updates des Typs <i>Anziehung</i> (vs. Random Walk) pro Iteration
<code>meanPairDistance[t]</code>	mittlere Euklid-Distanz aller Paarungspaare pro Iteration t
<code>pairFitnessGap[t]</code>	mittlerer Fitness-Abstand $ f_{male} - f_{female} $ pro Iteration
<code>interactionHistogram</code>	Häufigkeitsverteilung über die o. g. Update-Typen (Nuptial Dance, Männchen-Attraktion, Weibchen-Attraktion, Weibchen-Random-Walk)

1.4 Globales-Gedächtnis-Analyzer (7 P)

Metrik	Definition
<code>gbestTrajectory</code>	$\langle \text{iter}, \text{fitness} \rangle$ -Folge aller Iterationen; primärer Zweck: Nachvollziehbarkeit aller gbest-Verbesserungen und deren Quellen
<code>gbestUpdateCount</code>	Gesamtzahl gbest-Verbesserungen

Metrik	Definition
<code>gbestUpdateSourceDistribution</code>	Anteile {Male, Female, Offspring} an Updates — die Quelle wird zum Zeitpunkt des Updates erfasst (aus <code>moveMales</code> → MALE, aus <code>moveFemales</code> → FEMALE, aus <code>mate</code> → OFFSPRING), unabhängig davon, in welcher Pool-Liste der Agent nach der globalen Selektion landet
<code>improvementDelta[t]</code>	$\Delta = gbest_{t-1} - gbest_t$
<code>stagnationStreaks</code>	Liste der Längen aufeinanderfolgender Iterationen ohne gbest-Verbesserung ($\Delta = 0,0$ exakt; im Unterschied zu <code>plateauSegments</code> aus 1.6 ist hier ausschließlich $\delta = 0$ erlaubt)
<code>firstHittingIteration(\varepsilon)</code>	erste Iteration mit $gbest \leq \varepsilon$ (parametrierbar)

1.5 Lokales-Gedächtnis-Analyzer (6 P)

Metrik	Definition
<code>pbestUpdateCountPerAgent</code>	Map <i>Agent</i> → Anzahl <i>pbest</i> -Updates
<code>meanPbestImprovement</code>	mittlere relative Verbesserung über alle <i>pbest</i> -Updates; relative Verbesserung eines einzelnen Updates: $(pbest_{prev} - pbest_{new}) / pbest_{prev} $; Updates mit $pbest_{prev} = 0$ werden nicht in den Mittelwert einbezogen
<code>pbestPositionDiversity[t]</code>	mittlere komponentenweise Standardabweichung der <i>pbest</i> -Positionen aller Agenten pro Iteration t (d. h. pro Dimension d wird die Standardabweichung der Komponente d über alle Agenten berechnet; die D Werte werden arithmetisch gemittelt)
<code>pbestFitnessDistribution[t]</code>	min/q25/median/q75/max der <i>pbest</i> -Fitnesses pro Iteration

1.6 Konvergenz-Analyzer (5 P)

Metrik	Definition
<code>convergenceCurve</code>	beste Fitness pro Iteration, im Kontext des Konvergenz-Analyzers für semilogarithmische Visualisierung und Fitangepassung aufbereitet (konzeptuell analog zu <code>gbestTrajectory</code> in 1.4, jedoch eigenständig im Konvergenz-Analyzer gehalten, um die Trennung der Belange zu wahren)
<code>populationDiversity[t]</code>	mittlere paarweise Distanz oder Std.-Abw. der Positionen der $2 \times \text{populationSize}$ überlebenden Agenten nach globaler Selektion pro Iteration (nicht des temporären $4 \times$ großen Pools inkl. Offspring)

Metrik	Definition
<code>iterationsToThreshold(\varepsilon)</code>	Anzahl der benötigten Iterationen bis $g_{\text{best}} \leq \varepsilon$ — im Gegensatz zu <code>firstHittingIteration</code> (Aufg. 1.4) wird hier zusätzlich die <i>Konvergenzgeschwindigkeit</i> (Δt normiert auf Gesamtiterationen) als Begleitgröße ausgegeben
<code>plateauSegments</code>	Liste $(t_{\text{start}}, t_{\text{end}})$, in denen $\Delta g_{\text{best}} < \delta$ über $\geq k$ Iterationen (δ, k konfigurierbar; im Unterschied zu <code>stagnationStreaks</code> aus 1.4 ist $\delta > 0$ erlaubt)
<code>convergenceRateEstimate</code>	linearer Fit von $\log(g_{\text{best}})$ vs. Iteration über die letzten $N\%$ der Iterationen (N konfigurierbar, Default 20)

Aufgabe 2 — JUnit-5-Test-Suite (15 P)

Tests gehören in `src/test/java`. Es sind **AssertJ-Fluent-Assertions** zu verwenden (`org.assertj.core.api.Assertions.assertThat`).

2.1 Unit-Tests pro Analyzer (8 P)

- Pro Analyzer **mindestens 6 Testfälle**, davon mindestens
 - 1 Happy-Path-Test,
 - 1 Edge-Case (z. B. konstante Fitness, leere Population, sofortige Konvergenz),
 - 1 Test zu numerischer Stabilität (z. B. NaN-/Infinity-Behandlung).
- Determinismus: alle Tests verwenden einen festen Seed; keine Test-Flakiness.
- Verwendung von `@Nested`-Klassen je Analyzer.

2.2 Parametrisierte Tests (4 P)

- Mindestens **3** parametrisierte Tests mit unterschiedlichen Quellen (`@ValueSource`, `@CsvSource` und `@MethodSource` — jede Quelle mind. 1×).
- Mindestens 1 parametrisierter Test mit **dynamic display name** (`name = "..."`).

2.3 Test-Organisation, Lifecycle & Coverage (3 P)

- Sinnvolle Nutzung von `@BeforeEach` / `@BeforeAll` und Test-Fixtures.
- `@DisplayName` an jeder Top-Level-Testklasse.
- JaCoCo-Plugin im Build aktiviert; `jacoco.xml` wird beim Build erzeugt.
- **Branch-Coverage $\geq 80\%$** auf den Analytics-Klassen — wird automatisiert geprüft.

Phase 2 — mit KI-Unterstützung (50 P)

Aufgabe 3 — JGiven-Akzeptanztests (25 P)

3.1 BDD-Szenarien (12 P)

Mindestens **6 Szenarien** in Given/When/Then-Form mit JGiven-Stages. Pflicht-Szenarien:

#	Szenario (Kurzfassung)
AT-1	Reproduzierbarkeit: gleicher Seed $\Rightarrow$ gleiches gbest
AT-2	Konvergenz: nach 1000 Iterationen ist $\text{Ackley}(\text{gbest}) < 1 \cdot 10^{-3}$ (Smoke-Bound, mit Default-Konfiguration)
AT-3	Globales Gedächtnis: gbest-Trajektorie ist monoton nicht-steigend
AT-4	Agenten-Interaktion: Anteil weiblicher Anziehungen liegt über einer parametrisierten Schwelle
AT-5	Lokales Gedächtnis: jeder Agent erhält im Lauf mindestens 1 pbest-Update
AT-6	Plateau-Detektion: bei künstlich konstantem Fitness-Stream wird genau 1 Plateau erkannt

Jedes Szenario muss mindestens **drei** Stages durchlaufen (Given..., when..., Then...) und mit @Description versehen sein.

3.2 Stage-Architektur (6 P)

- Drei wiederverwendbare Stage-Klassen: GivenMayflyConfiguration, whenAlgorithmRuns, ThenAnalyticsReport.
- Stages erben von Stage<...> und nutzen @ProvidedScenarioState / @ExpectedScenarioState korrekt.
- Fluent Self-Type-Pattern (SELF) ist sauber implementiert.

3.3 JGiven-HTML-Report (4 P)

- HTML-Report wird beim Build automatisch unter target/jgiven-reports/html/ erzeugt.
- Tags @Tag("agent-interaction"), @Tag("global-memory"), @Tag("local-memory"), @Tag("convergence") werden im Report sichtbar.
- README enthält Screenshot/Verweis.

3.4 Multi-Run-Akzeptanztests (3 P)

- Mindestens 1 Szenario, das **N (≥ 10) Läufe** mit verschiedenen Seeds aggregiert und Erwartungen über *Mittelwert $\pm$ Standardabweichung* der finalen Fitness formuliert.

Aufgabe 4 — Reporting, Export & statistische Auswertung (15 P)

4.1 Daten-Export (5 P)

- `AnalyticsExporter` mit zwei Implementierungen: `CsvExporter` und `JsonExporter`.
- Das Interface `AnalyticsExporter` deklariert mindestens die Methode `void export(AnalyticsReport report, java.io.Writer out) throws java.io.IOException`.
- JSON ohne Drittbibliothek selbst zu serialisieren (kleiner, eigener Writer ist erlaubt).
- Schema im Architektur-Dokument dokumentieren.

4.2 Markdown-Report-Generator (6 P)

`MarkdownReportGenerator` erzeugt einen Report mit:

- Header (Konfiguration, Seed, Tool-Versionen, Lauf-Zeitstempel),
- Sektion pro Analyzer mit Schlüsselmetriken in Tabellenform,
- Unicode-Sparkline der gbest-Kurve (z. B. , Block-Zeichen U+2581–U+2588),
- Mermaid-Diagramm der Konvergenzkurve **oder** Plateau-Segmente.

4.3 Statistische Auswertung über mehrere Läufe (4 P)

- Klasse `MultiRunStatistics`: Mittelwert, Median, Std.-Abw., 25-/75-Quartil, Konfidenzintervall (95 %, t-Verteilung — Tabellenwerte erlaubt).
- Aggregat-Report wird in den Markdown-Report eingebettet.

Aufgabe 5 — Dokumentation & KI-Reflexion (10 P)

5.1 Architektur-Dokumentation (4 P) — `docs/architecture.md`

- Komponenten-Übersicht mit Mermaid-Klassendiagramm.
- Sequenzdiagramm (Mermaid) eines Iterationsschritts inkl. Listener-Aufrufe.
- ADR (Architecture Decision Records) für mindestens 3 Entscheidungen (z. B. „warum sealed interface für Events“, „warum kein Streaming-Export“).

5.2 README & Build-Anweisungen (2 P) — `README.md`

- Build- und Run-Befehle (Linux & Windows).
- JDK-Verifikation (`java -version` -Hinweis).
- Verzeichnisstruktur, Coverage-Report-Pfad, JGiven-Report-Pfad.
- Hinweis auf das Maven-Profil `multi-run` (bzw. Gradle-Task `multiRunVerify`) für `mvn -Pmulti-run verify` — das Profil/der Task muss in der Build-Datei definiert sein.

5.3 KI-Reflexion (4 P) — `docs/ai-usage-log.md`

- Tool-Identifikation (Name, Version, Datum).
- Chronologisches Prompt-Log, **alle** in Phase 2 verwendeten Prompts.
- Markierung erfolgreicher / fehlerhafter Prompts.
- Reflexion (≥ 400 Wörter): Welche Aufgaben hat KI gut gelöst, wo war Nacharbeit nötig, welche Halluzinationen sind aufgetreten, wie wurden sie erkannt?

6. Punkteübersicht

Phase	Aufgabe	Teilaufgabe	Punkte	Kumuliert
1	1. Refactoring & Analytics	1.1 Refactoring	5	5
1		1.2 Analytics-Infrastruktur	5	10
1		1.3 Agenten-Interaktion	7	17
1		1.4 Globales Gedächtnis	7	24
1		1.5 Lokales Gedächtnis	6	30
1		1.6 Konvergenz	5	35
1	2. JUnit-5-Test-Suite	2.1 Unit-Tests	8	43
1		2.2 Parametrisierte Tests	4	47
1		2.3 Test-Organisation & Coverage	3	50
2	3. JGiven-Akzeptanztests	3.1 BDD-Szenarien	12	62
2		3.2 Stage-Architektur	6	68
2		3.3 HTML-Report	4	72
2		3.4 Multi-Run	3	75
2	4. Reporting & Statistik	4.1 Export	5	80
2		4.2 Markdown-Generator	6	86
2		4.3 Multi-Run-Statistik	4	90
2	5. Dokumentation & KI-Reflexion	5.1 Architektur	4	94
2		5.2 README	2	96
2		5.3 KI-Reflexion	4	100

Phase 1: 50 P • Phase 2: 50 P • Gesamt: 100 P

7. Bewertungs- und Qualitätskriterien

Jede Teilaufgabe wird über vier Dimensionen bewertet. Die Gewichtung pro Dimension ist **aufgabenspezifisch**, summiert aber stets auf 100 % der Teilpunktzahl.

Dimension	Indikatoren
Funktionale Korrektheit	Anforderungen erfüllt; Verhaltensgleichheit zur Basis (Aufg. 1.1); Tests grün; Reports plausibel
Code-Qualität	SOLID, klare Namen, kohärente Pakete, JavaDoc an öffentlichen APIs, sinnvolle Lombok-Nutzung, moderne Java-25-Features (<code>var</code> , <code>Math.clamp</code> , <code>record</code> , sealed types, pattern matching, sequenced collections) — <i>ohne Preview</i>
Test-Qualität	AAA bzw. Given/When/Then, Determinismus, Isolation, sprechende Testnamen, sinnvolle Edge-Cases, Coverage-Schwelle
Dokumentations-Qualität	Markdown-Formatierung, korrekte Mermaid-Diagramme, ADRs, vollständiges KI-Log

Allgemeine Qualitätskriterien

- **Build-Reproduzierbarkeit:** `mvn verify` bzw. `gradle build` läuft ohne Anpassungen durch.
- **Statische Analyse:** Es sind keine Compiler-Warnungen (außer dokumentierten `@SuppressWarnings`-Stellen mit Begründung) zulässig.
- **Determinismus:** Jeder Lauf mit festem Seed muss bitgenau reproduzierbar sein.
- **Single-Threaded:** Verstöße gegen die Concurrency-Sperre führen zur Aberkennung von 5 P.
- **Lombok-Konformität:** Verwendung von `@Data`, `@Value`, `@Builder`, `@Getter`, `@Setter`, `@AllArgsConstructor`, `@NoArgsConstructor`, `@RequiredArgsConstructor` ist erlaubt.
Nicht erlaubt: `@SuperBuilder`, `@FieldNameConstants`, `@ExtensionMethod`, `@UtilityClass`, `@Helper`, `lombok.experimental.*`.

Punktabzüge (Maluskatalog)

Verstoß	Abzug
Verwendung von <code>--enable-preview</code>	-5 P
Concurrency / parallele Streams	-5 P
Nicht erlaubte Drittbibliothek	-5 P pro Bibliothek
Compiler-Warnung ohne Begründung	-0,5 P pro Warnung (Deckel -3 P)
Fehlendes oder unvollständiges KI-Log in Phase 2	bis zu -10 P
KI-Nutzung in Phase 1 (nachgewiesen)	0 P für die gesamte Prüfungsleistung

8. Abgabeformat

Verzeichnisstruktur (verbindlich)

```
mayfly-analytics/  
├─ pom.xml                      (oder build.gradle.kts)  
├─ README.md  
├─ HONOR_DECLARATION.md        ← Vorlage für die Selbstverpflichtung Phase 1  
├─ HONOR_DECLARATION_signed.pdf ← eingescannte/fotografierte Unterschrift (Pflicht)  
├─ docs/  
│   ├─ architecture.md  
│   ├─ analytics-report.md      ← generiert, beigelegt  
│   └─ ai-usage-log.md  
├─ src/  
│   ├─ main/java/<package>/...  
│   └─ test/java/<package>/...  
└─ target/                     ← darf entfernt sein, Build muss sie erzeugen können  
    ├─ site/jacoco/             ← JaCoCo-Coverage  
    └─ jgiven-reports/html/     ← JGiven-Report
```

Abgabeartefakt

Ein **ZIP-Archiv** `[matrikelnummer].zip` mit dem o. g. Inhalt
(ohne `target/`, ohne `.idea/`, ohne `node_modules`-Äquivalente).
Die Git-Historie ist als `git-bundle` oder `.git`-Ordner beizulegen.

Pflicht-Befehle, die laufen müssen

Befehl	Erwarteter Effekt
<code>mvn verify</code> (oder <code>gradle build</code>)	Compile, alle Tests grün, JaCoCo + JGiven erzeugt
<code>mvn -Pmulti-run verify</code>	Multi-Run-Statistik wird generiert

9. Wichtige Hinweise

- Eigenständigkeit.** Phase 1 ist *vollständig* eigenständig zu bearbeiten. Auch das Lesen fremden Codes (außer der gestellten Basis und der offiziellen Dokumentation der zugelassenen Bibliotheken) ist in Phase 1 untersagt.
- Aufgaben-Reihenfolge.** Phase 1 ist vor Phase 2 abzuschließen. Nachträgliche Änderungen an Phase-1-Code in Phase 2 sind erlaubt, aber im KI-Log gesondert auszuweisen.
- Reproduzierbarkeit.** Alle Zufallsanteile sind über einen Seed steuerbar zu machen. Tests, die von Wall-Clock-Zeit abhängen, sind unzulässig.
- Determinismus + Concurrency.** Auch wenn JUnit 5 Tests parallel ausführen *könnte*, ist `junit-platform.properties` so zu konfigurieren, dass Tests **sequenziell** laufen.
- Plagiate.** Identische Lösungen (auch in Teilen) führen für alle beteiligten Studierenden zur Bewertung mit 0 Punkten.

6. **Fragen während der Bearbeitung.** Inhaltliche Rückfragen zur Aufgabenstellung sind erlaubt (Forum). Fragen zur Lösung sind unzulässig.

Anhang A — Glossar

Begriff	Bedeutung im Kontext
Agent	einzelne Mayfly (Männchen oder Weibchen)
gbest	global beste je gefundene Position (globales Gedächtnis)
pbest	individuell beste je gefundene Position eines Agenten (lokales Gedächtnis)
Konvergenz	Annäherung der besten Fitness an das (vermutete) Optimum
Plateau	aufeinanderfolgende Iterationen ohne signifikante Verbesserung
Nuptial Dance	stochastischer Random-Walk-Update des besten Männchens
Sparkline	textbasierte Visualisierung einer Zeitreihe mittels Unicode-Block-Zeichen (z. B. ); kein ASCII sondern Unicode-Codepoints U+2581–U+2588
Listener / Observer	Empfänger von Algorithmus-Events ohne Rückwirkung auf den Lauf
ADR	Architecture Decision Record; kurzes, datiertes Entscheidungs-Protokoll

Ende der Spezifikation. Stand: 10.05.2026 (final)



Prof. Dr. Carsten Müller
2026.05.10 11:40:00 +02'00